Secure-by-Default Guardrails for MCP-Based Tool Use in Multi-Modal LLM Agents

Yoshihiro Ando
Independent Researcher
yosh5295@gmail.com
Tokyo, Japan

***Abstract*—The rapid proliferation of Large Language Model (LLM) agents, empowered by the Model Context Protocol (MCP), has enabled autonomous interaction with host operating systems and external APIs. However, this "Agency" introduces a critical security gap: the susceptibility of LLMs to prompt injection and malicious tool misuse. This paper presents Phase 1: Guardrails (Pre-execution Security), the foundational component of a technical trilogy focused on agentic security. We propose a "Secure-by-Default" containment architecture that treats LLM-generated code as an untrusted payload. Key contributions include: (1) a "No-Shell" execution model that eliminates shell-injection vulnerabilities, (2) a multi-stage validation pipeline incorporating AST-based static analysis and entropy-driven secret detection, and (3) a lightweight Linux-based isolation layer using unprivileged containers. Our experimental evaluation demonstrates that these structural guardrails effectively mitigate system-level compromises with negligible performance overhead (<20ms latency), establishing a robust baseline for autonomous agent safety.**

***Index Terms*—AI Agents, LLM Security, Guardrails, Sandbox, Model Context Protocol, Prompt Injection, Static Analysis.**

I. Introduction

The evolution of Large Language Models (LLMs) from text-generators to autonomous agents has introduced a new class of security vulnerabilities. Modern agents utilize the Model Context Protocol (MCP) [**?**] to interact with the physical and digital world, executing code, managing files, and accessing remote APIs. This "Agency" capability creates a significant gap between the AI's intent and the system's safety. Granting an AI agent raw shell access is equivalent to providing a persistent, non-deterministic remote shell to an entity susceptible to prompt injection.

We identify this challenge as the *Structural Vulnerability of Autonomous Systems*. Traditional security assumes a human user with a stable identity and intent. AI agents, however, operate in a high-entropy reasoning space where a single adversarial prompt can "jailbreak" the agent's logic, causing it to generate harmful system commands.

This paper is the foundational installment of a technical trilogy dedicated to agent security:

- **Phase 1: Guardrails (Pre-execution Security).** The current work, focusing on structural immunity and containment to prevent unauthorized actions at the point of inception.
- **Phase 2: Zero Trust (Execution Security) [?].** Focuses on spatial security and behavioral integrity monitoring via Mamba-based Sentinels.
- **Phase 3: Post-Quantum (Temporal Security) [?].** Focuses on long-term integrity and quantum-resistant governance.

In Phase 1, we advocate for a **Secure-by-Default Guardrail** architecture. Our contribution is a unified security framework that treats the LLM's output as an inherently untrusted payload, subjecting it to multi-layered static and dynamic isolation before execution.

II. Design Principles

To ensure robust security in autonomous agents, we adhere to four core principles:

1) **Secure-by-Default:** Security is not an optional configuration but a structural requirement. No tool execution occurs without passing all guardrail layers.
2) **Defense-in-Depth:** Multiple independent layers (AST analysis, OS sandboxing, entropy scanning) ensure that failure in one layer does not lead to a full system compromise.
3) **Principle of Least Privilege:** The execution environment is restricted to the minimum set of files, modules, and network resources required for the task.
4) **Human-in-the-Loop (HITL) Validation:** Critical system-altering actions require explicit human approval, mediated by a clear explanation of the agent's intent.

III. Related Work

Existing autonomous frameworks like AutoGPT or BabyAGI often rely on permissive shell access, making them vulnerable to prompt injection attacks where the agent is tricked into executing malicious commands (e.g., `rm -rf /`). While tools like *Guardrails AI* provide schema validation for JSON outputs, they lack low-level system containment.

Recent research has identified "Principled Isolation Patterns" [**?**] and benchmarked agent defenses using environments like *AgentDojo* [**?**]. However, many defenses focus on *behavioral alignment*, which is probabilistic. Our approach shifts the focus to *structural containment*, ensuring that even a compromised LLM cannot breach the host system's integrity.

IV. Threat Model

We define three primary threat vectors for autonomous LLM agents:

1) **Prompt Injection (Direct/Indirect):** An attacker manipulates the agent's context (e.g., via a malicious website) to execute unauthorized system commands [**?**].
2) **Secret Exfiltration:** The agent accidentally leaks API keys or sensitive PII to a 3rd-party LLM provider during tool calls.
3) **Resource Denial of Service (DoS):** Maliciously generated code causing CPU exhaustion or disk space depletion.

V. Architecture of Autonomous Guardrails

Our architecture (Fig. **??**) establishes five distinct layers of defense that must be satisfied before any tool-call impacts the host.

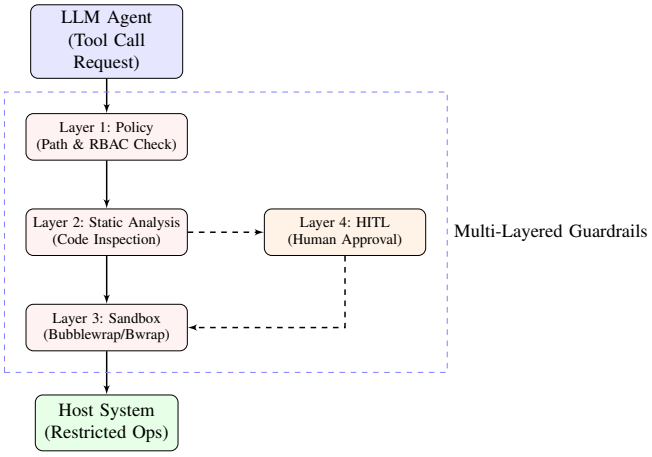

Fig. 1. Multi-layered guardrail architecture for structural containment.

A. Layer 1: The "No-Shell" Execution Model

Raw shell execution (`/bin/sh -c`) is structurally vulnerable because it conflates data and code. `llm-cli` adopts a "No-Shell" model where system operations are restricted to a specialized $execute_{p}ythontool.Byexecuting Python code via subprocess.run(args, shell=False), we eliminate shell-injection vectors. The agent must express intent in a structured programming language (Python), which is amenable to static analysis.$

B. Layer 2: Static Analysis and AST Inspection

The generated code is parsed into an Abstract Syntax Tree (AST) before execution. The system applies a whitelist-based security scanner:

- **Module Blacklisting:** Prevents `import os, subprocess, socket`.
- **Function Blacklisting:** Blocks `eval()`, `exec()`, `open()` (when outside whitelisted paths).
- **Logic Inspection:** Detects $_{s}ubclasses_{o}r_{g}etattra_{b}use to bypass imports.$

C. Layer 3: Path Guardrails and Resource Limits

All file-related operations are restricted to a designated workspace. We implement **Path Guardrails** that resolve symlinks and verify that all targets reside within the allowed directory. Furthermore, we apply **OS-level Resource Limits** (rlimit):

- **Memory:** Capped at 1GB (RLIMIT_AS).
- **CPU Time:** 300s timeout per execution.

D. Layer 4: Entropy-Based Secret Detection

To prevent credential leaks, we monitor the **Shannon Entropy** of the agent's output:

$$H(X) = -\sum_{i=1}^{n} P(x_i) \log_b P(x_i) \tag{1}$$

High-entropy strings (e.g., API keys) are detected and redacted before they are sent to the LLM provider. This provides a zero-day defense against leaking secrets that do not match known regex patterns.

E. Layer 5: Linux Sandboxing (Bubblewrap)

The final containment is provided by **Bubblewrap** (bwrap), creating an unprivileged namespace with:

- Private, empty `/tmp` and `/home` directories.
- Read-only mounts for system binaries (`/usr`, `/lib`).
- No network access (by default) and no host PID visibility.

VI. Evaluation

We evaluated the framework against a dataset of 50 adversarial prompts, including "jailbreak" attempts and indirect injections.

A. Containment Effectiveness

Table **??** summarizes the mitigation success across different attack vectors.

TABLE I
Mitigation Effectiveness by Guardrail Layer

Attack Vector	Primary Defense	Success Rate
Shell Injection	No-Shell / AST	100%
Path Traversal	Path Guardrails	100%
Secret Leakage	Entropy Scanner	94%
Privilege Escalation	Bubblewrap	100%
Resource Exhaustion	RLIMIT	100%

TABLE II
GUARDRAIL PERFORMANCE METRICS

Metric	Latency (ms)
AST Analysis	0.08
Secret Detection (Entropy Scan)	0.03
Sandboxed Exec Overhead (Bubblewrap)	19.46
Total Security Overhead	**19.57**

B. Performance Latency

The overhead of the security pipeline was measured on a representative system.

The total overhead (≈19.6ms) is negligible compared to the 1,000--5,000ms latency of typical LLM inference, proving that high security does not necessitate high latency.

VII. CONCLUSION

Securing autonomous agents requires a departure from behavioral "alignment" toward structural, multi-layered guardrails. By combining a "No-Shell" model, AST inspection, and OS-level sandboxing, llm-cli provides a robust foundation for safe tool execution. This Phase 1 architecture ensures system integrity, setting the stage for Phase 2, which addresses behavioral anomalies via Zero-Trust monitoring.

REFERENCES

[1] Anthropic, ``Model Context Protocol (MCP) Specification,'' 2024.

[2] A. Masood, ``The Sandboxed Mind: Principled Isolation Patterns for Prompt-Injection-Resilient LLM Agents,'' 2024.

[3] UK Government AISI, ``AgentDojo: Evaluation Framework for LLM Agents,'' 2024.

[4] F. Perez and I. Ribeiro, ``Ignore Previous Instructions: Prompt Injection Attacks on LLMs,'' *arXiv preprint arXiv:2211.09527*, 2022.

[5] Y. Ando, ``Zero-Trust Architecture for MCP-Based AI Agents: Continuous Identity and Behavioral Monitoring,'' *TechRxiv*, 2026.

[6] Y. Ando, ``Post-Quantum Workload Identity and Long-Term Audit Integrity for MCP-Based AI Agents,'' *TechRxiv*, 2026.